1. Imports

In [6]:
```
import numpy as np
import pandas as pd
from imblearn.combine import SMOTETomek
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_sp
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
import xgboost as xgb
import lightgbm as lgb
from sklearn.metrics import (classification_report, confusion_matrix, roc_auc_sc
                            precision_recall_curve, average_precision_score,
                            precision_score, recall_score, f1_score, roc_curve)
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.inspection import permutation_importance
import warnings
warnings.filterwarnings('ignore')
```

2. Load Dataset

In [7]:
```
df = pd.read_csv('C:/Users/PC\Desktop/Insurance assistant/data/synthetic_insuran
df.head()
```

Out[7]:

	subscription_length	vehicle_age	customer_age	region_density	airbags	displacemen
0	9.3	1.2	41	8794	6	149
1	8.2	1.8	35	27003	2	149
2	9.5	0.2	44	8794	6	149
3	5.2	0.4	44	73430	2	79
4	10.1	1.0	56	5410	2	149

3. Basic Information about the Dataset

In [8]:
```
df.columns
df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 109688 entries, 0 to 109687
Data columns (total 13 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   subscription_length  109688 non-null  float64
 1   vehicle_age          109688 non-null  float64
 2   customer_age         109688 non-null  int64
 3   region_density       109688 non-null  int64
 4   airbags              109688 non-null  int64
 5   displacement         109688 non-null  int64
 6   cylinder             109688 non-null  int64
 7   turning_radius       109688 non-null  float64
 8   length               109688 non-null  int64
 9   width                109688 non-null  int64
 10  gross_weight         109688 non-null  int64
 11  ncap_rating          109688 non-null  int64
 12  claim_status         109688 non-null  int64
dtypes: float64(3), int64(10)
memory usage: 10.9 MB
```

Out[8]:

	subscription_length	vehicle_age	customer_age	region_density	airbags
count	109688.000000	109688.000000	109688.000000	109688.000000	109688.000000
mean	6.737890	1.321939	44.891301	18231.322925	3.149278
std	3.990381	1.047179	6.831500	16916.035608	1.834063
min	0.000000	0.000000	35.000000	290.000000	1.000000
25%	2.974492	0.400000	40.000000	6112.000000	2.000000
50%	7.007112	1.177702	44.000000	8794.000000	2.000000
75%	10.600000	2.000000	49.000000	27003.000000	6.000000
max	14.000000	20.000000	75.000000	73430.000000	6.000000

4. Missing Values

In [10]:
```
df.isnull().sum()
missing_percent = df.isnull().mean() * 100
missing_percent[missing_percent > 0].sort_values(ascending=False)
```

Out[10]:
```
Series([], dtype: float64)
```

5. Date cleaning

In [9]:
```
df.drop_duplicates(inplace=True)
```

Check for missing data

In [11]:
```
df.isnull().sum()
```

Out[11]:
```
subscription_length    0
vehicle_age            0
customer_age           0
region_density         0
airbags                0
displacement           0
cylinder               0
turning_radius         0
length                 0
width                  0
gross_weight           0
ncap_rating            0
claim_status           0
dtype: int64
```

Ckeck data types

In [12]:
```
df.dtypes
```

Out[12]:
```
subscription_length    float64
vehicle_age            float64
customer_age             int64
region_density           int64
airbags                  int64
displacement             int64
cylinder                 int64
turning_radius         float64
length                   int64
width                    int64
gross_weight             int64
ncap_rating              int64
claim_status             int64
dtype: object
```

Renaming columns

In [13]:
```
rename_dict = {
    'subscription_length': 'sub_length',
    'region_code': 'region',
    'max_torque': 'max_torque_nm',
    'max_power': 'max_power_hp',
    'is_esc': 'esc',
    'is_adjustable_steering': 'adjustable_steering',
    'is_tpms': 'tpms',
    'is_parking_sensors': 'parking_sensors',
    'is_parking_camera': 'parking_camera',
    'rear_brakes_type': 'rear_brakes',
    'transmission_type': 'transmission',
    'steering_type': 'steering',
    'is_ecw': 'ecw',
    'is_speed_alert': 'speed_alert',
    'ncap_rating': 'safety_rating',
}

df.rename(columns=rename_dict, inplace=True)
```

In [14]:
```
df.columns
```

Out[14]:
```
Index(['sub_length', 'vehicle_age', 'customer_age', 'region_density',
       'airbags', 'displacement', 'cylinder', 'turning_radius', 'length',
       'width', 'gross_weight', 'safety_rating', 'claim_status'],
      dtype='object')
```

In [15]:
```
df.dtypes
```

Out[15]:
```
sub_length        float64
vehicle_age       float64
customer_age        int64
region_density      int64
airbags             int64
displacement        int64
cylinder            int64
turning_radius    float64
length              int64
width               int64
gross_weight        int64
safety_rating       int64
claim_status        int64
dtype: object
```

In [16]:
```
df_object = df.select_dtypes(include='object')
df_object.head()
```

Out[16]:

0
1
2
3
4

Remove non-numeric chars and convert to floats

In [17]:
```
df["max_torque_value"] = df["max_torque_nm"].str.extract(r'(\d+\.?\d*)Nm').astyp
df['max_torque_rpm'] = df['max_torque_nm'].str.extract(r'(\d+\.?\d*)rpm').astype

df['max_power_value'] = df['max_power_hp'].str.extract(r'(\d+\.?\d*)bhp').astype
df['max_power_rpm'] = df['max_power_hp'].str.extract(r'(\d+\.?\d*)rpm').astype(f

df.drop(['max_torque_nm', 'max_power_hp'], axis=1, inplace=True)

bool_cols = ['esc', 'adjustable_steering', 'tpms', 'parking_sensors', 'parking_c
for col in  bool_cols:
    df[col] = df[col].map({'Yes': 1  , 'No': 0})

cat_cols = ['region' , 'segment', 'model', 'fuel_type', 'engine_type', 'transmis
for col in cat_cols:
    df[col] = df[col].astype('category')

df.drop('policy_id', axis=1, inplace=True)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File c:\Users\PC\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas
\core\indexes\base.py:3812, in Index.get_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyOb
jectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyOb
jectHashTable.get_item()

KeyError: 'max_torque_nm'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[17], line 1
----> 1 df["max_torque_value"] = df[               ].str.extract(r'(\d+\.?\d*)N
m').astype(float)
      2 df['max_torque_rpm'] = df['max_torque_nm'].str.extract(r'(\d+\.?\d*)rp
m').astype(float)
      4 df['max_power_value'] = df['max_power_hp'].str.extract(r'(\d+\.?\d*)bh
p').astype(float)

File c:\Users\PC\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas
\core\frame.py:4107, in DataFrame.__getitem__(self, key)
   4105 if self.columns.nlevels > 1:
   4106     return self._getitem_multilevel(key)
-> 4107 indexer = self.columns.get_loc(key)
   4108 if is_integer(indexer):
   4109     indexer = [indexer]

File c:\Users\PC\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas
\core\indexes\base.py:3819, in Index.get_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'max_torque_nm'
```

In [50]:
```
df.dtypes
```

Out[50]:
```
sub_length        float64
vehicle_age       float64
customer_age        int64
region_density      int64
airbags             int64
displacement        int64
cylinder            int64
turning_radius    float64
length              int64
width               int64
gross_weight        int64
safety_rating       int64
claim_status        int64
dtype: object
```

In [18]:
```
bool_like_cols = []
for col in df.select_dtypes(include='object').columns:
    unique_vals = df[col].dropna().unique()
    if set(unique_vals).issubset({'Yes', 'No'}):
         bool_like_cols.append(col)

print("Boolean-like columns still as object:", bool_like_cols)
```

```
Boolean-like columns still as object: []
```

In [19]:
```
for col in bool_like_cols:
    df[col] = df[col].map({'Yes': 1, 'No': 0})
```

In [20]:
```
df.head()
```

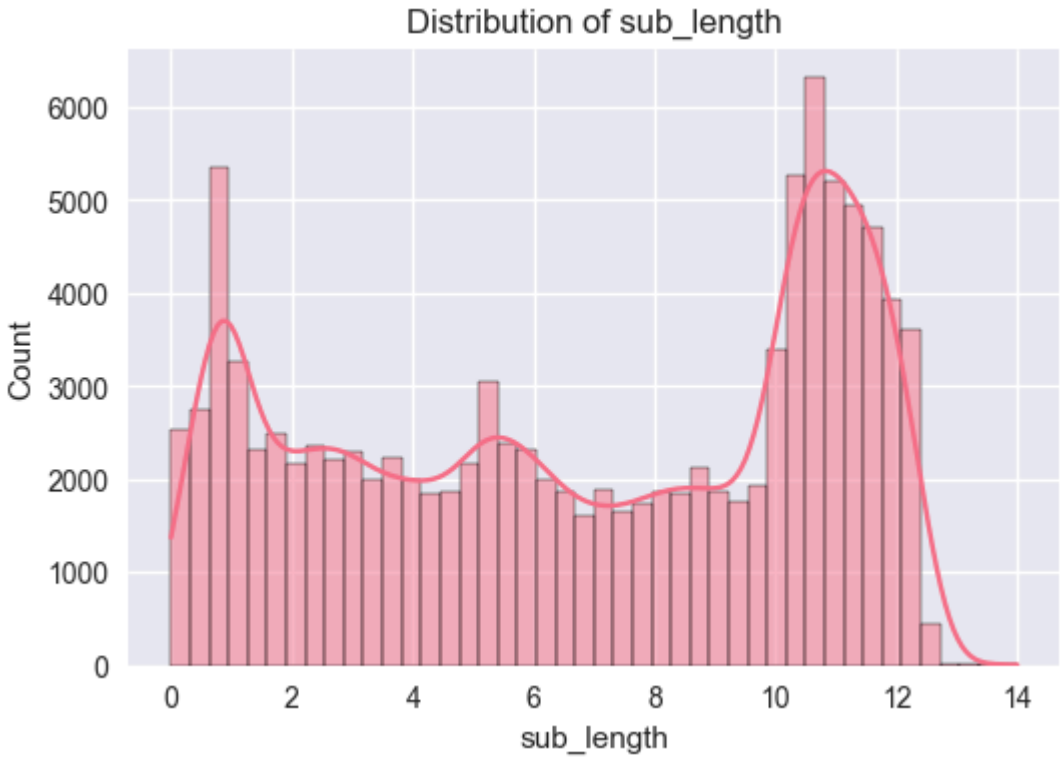
Out[20]:

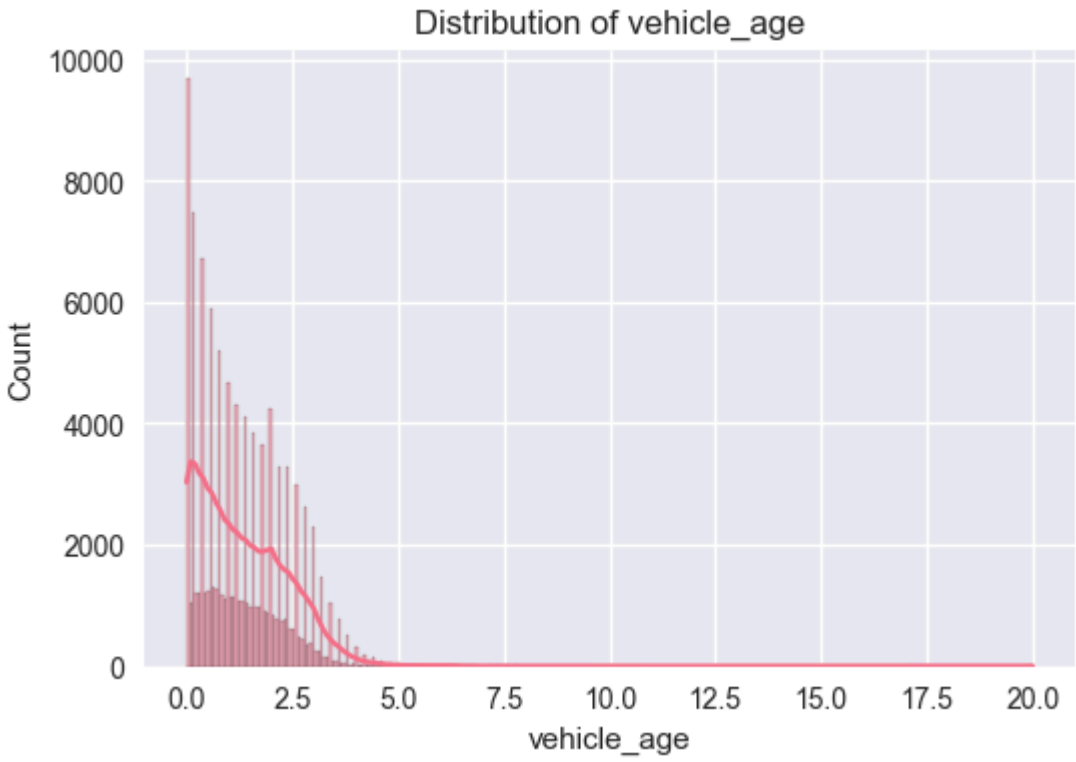
	sub_length	vehicle_age	customer_age	region_density	airbags	displacement	cylind
0	9.3	1.2	41	8794	6	1493	
1	8.2	1.8	35	27003	2	1498	
2	9.5	0.2	44	8794	6	1493	
3	5.2	0.4	44	73430	2	796	
4	10.1	1.0	56	5410	2	1497	

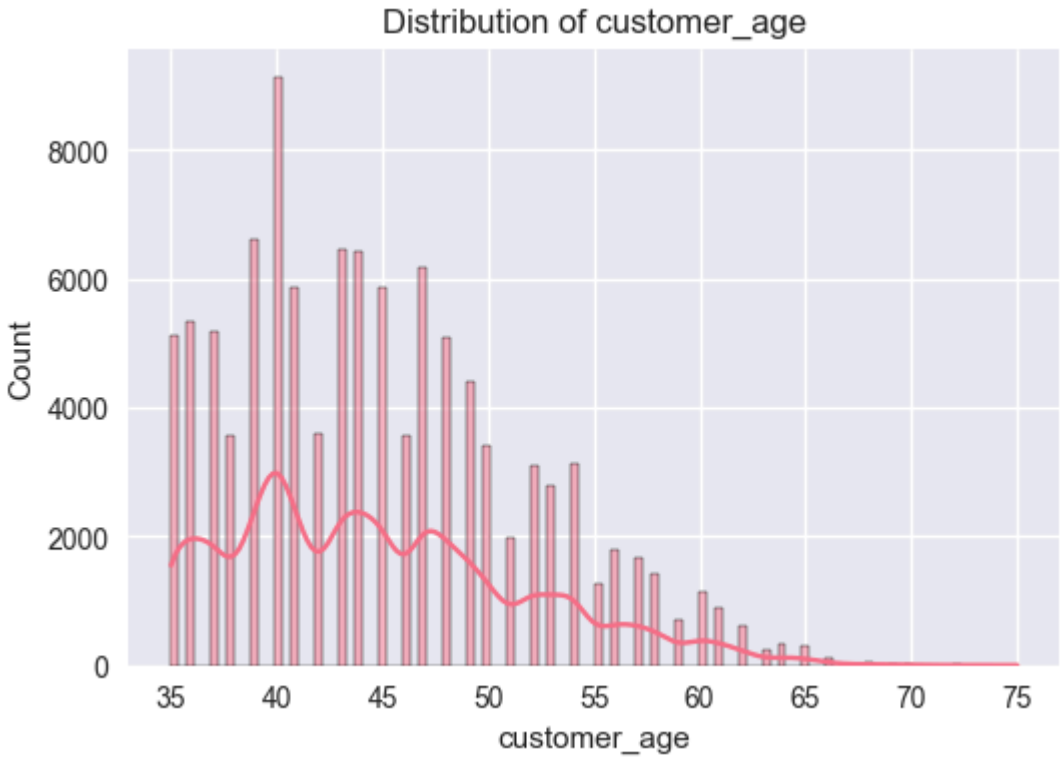
In [21]:
```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols=df.select_dtypes(include=['category']).columns.tolist()
```

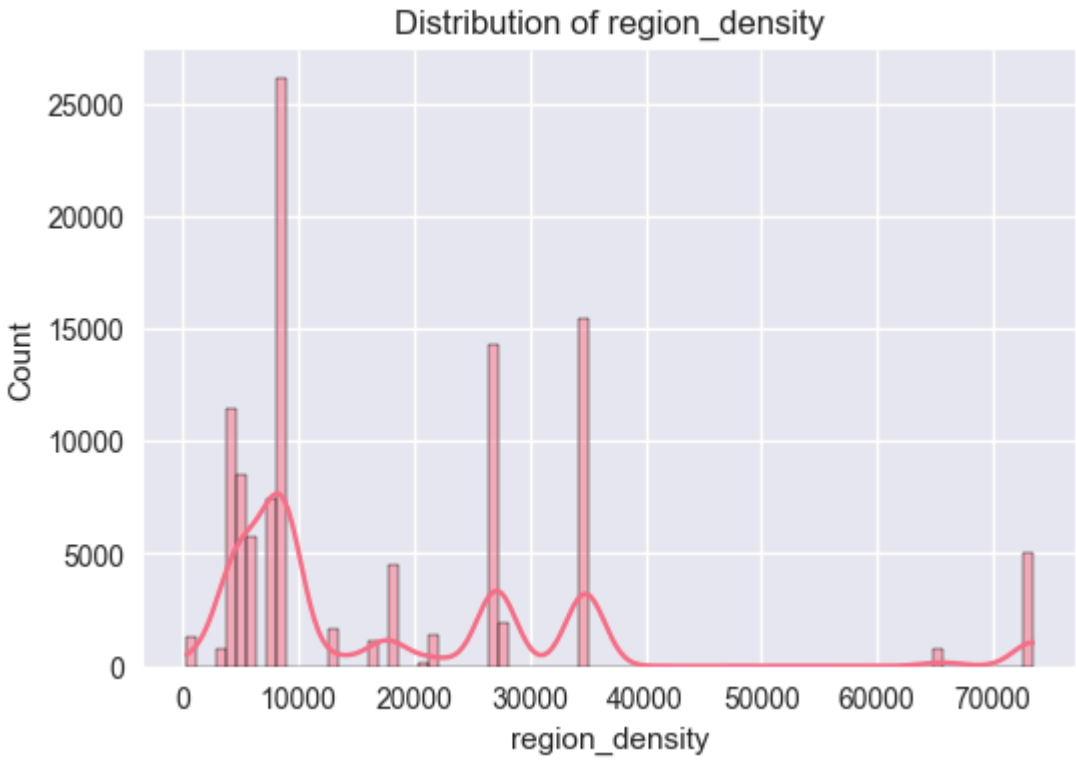
Data visualization

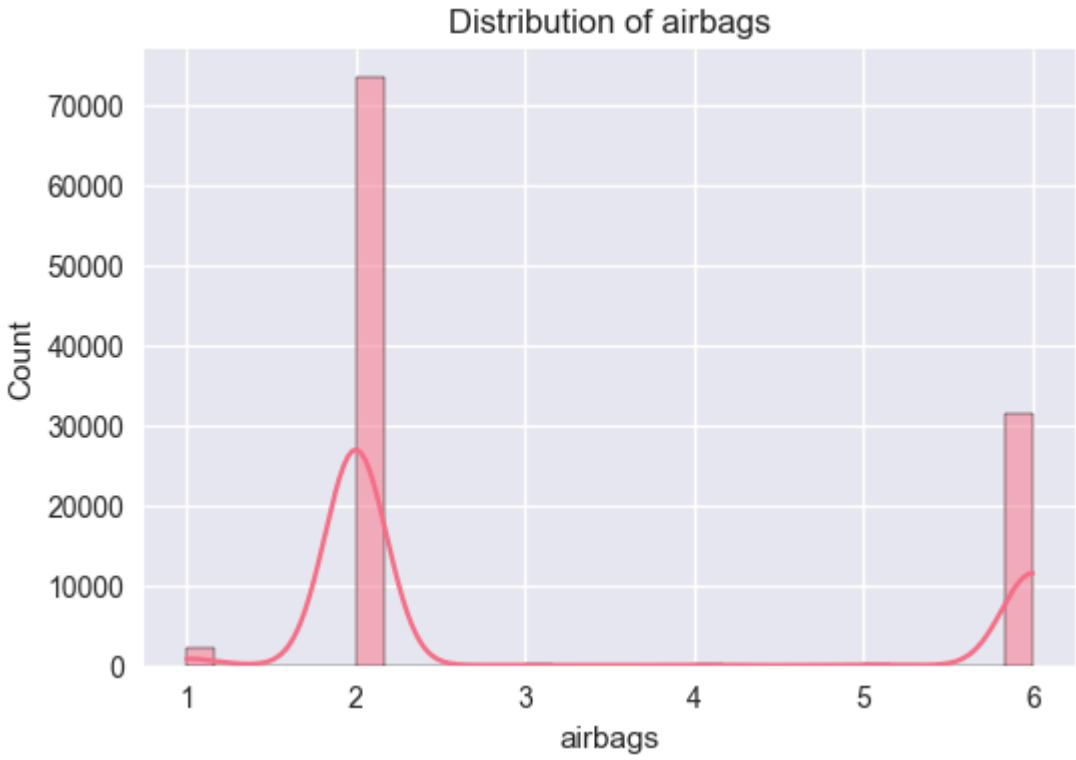
In [22]:
```
for col in num_cols :
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.show()
```

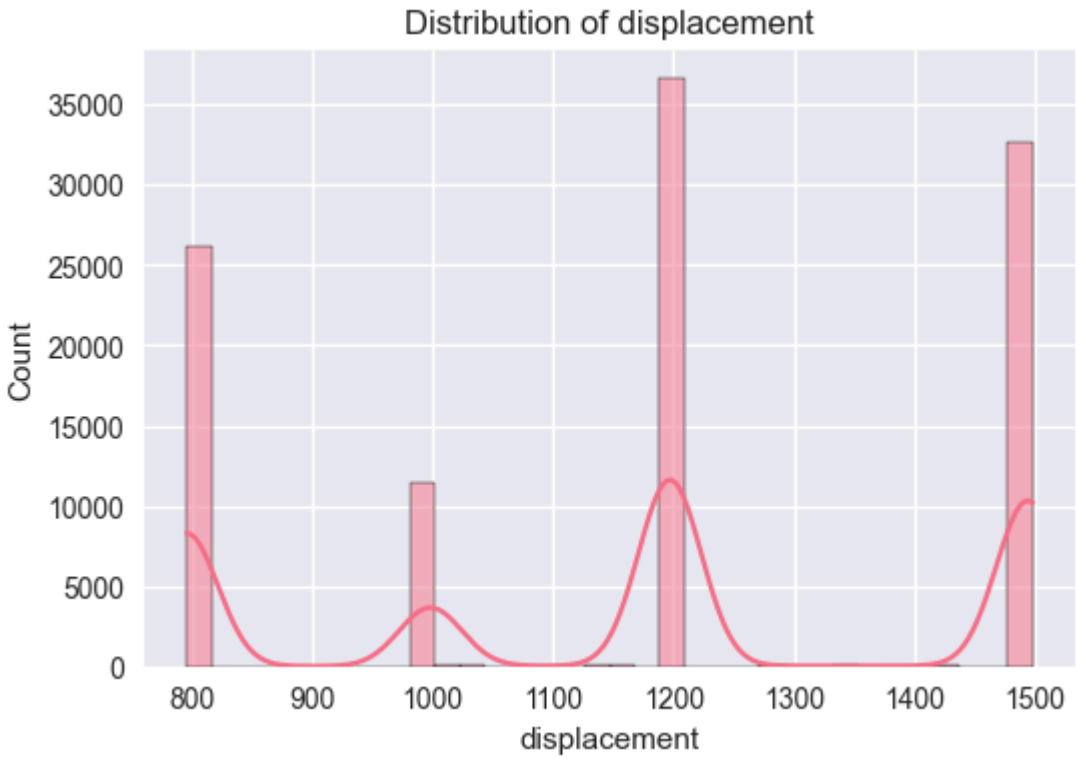

Distribution of sub_length
Count
6000
5000
4000
3000
2000
1000
0
0
2
4
6
8
10
12
14
sub_length

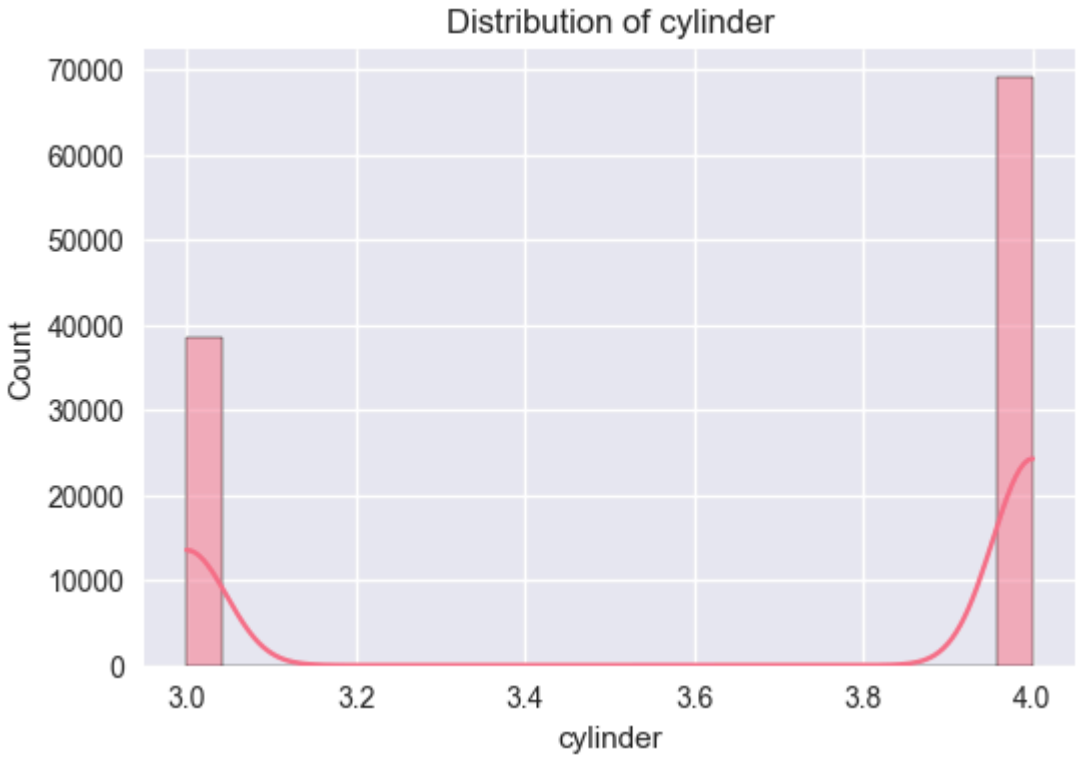

Distribution of vehicle_age
Count
10000
8000
6000
4000
2000
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
vehicle_age

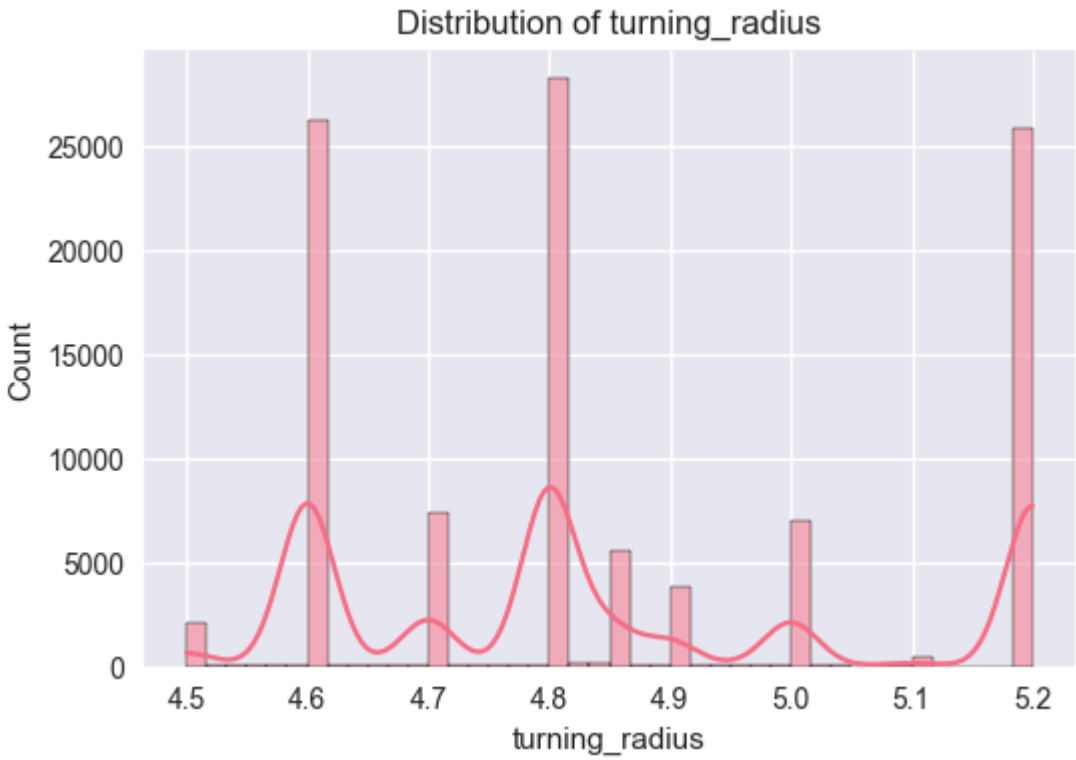

Distribution of customer_age
Count
8000
6000
4000
2000
0
35
40
45
50
55
60
65
70
75
customer_age

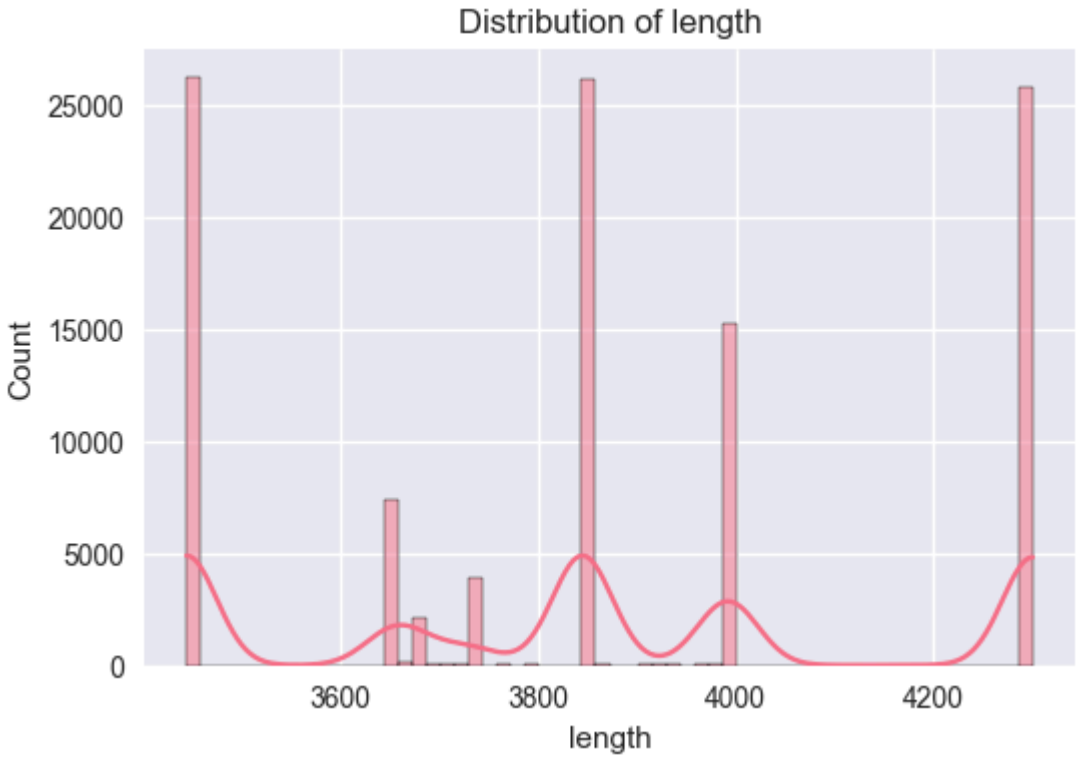

Distribution of region_density
Count
25000
20000
15000
10000
5000
0
0
10000
20000
30000
40000
50000
60000
70000
region_density

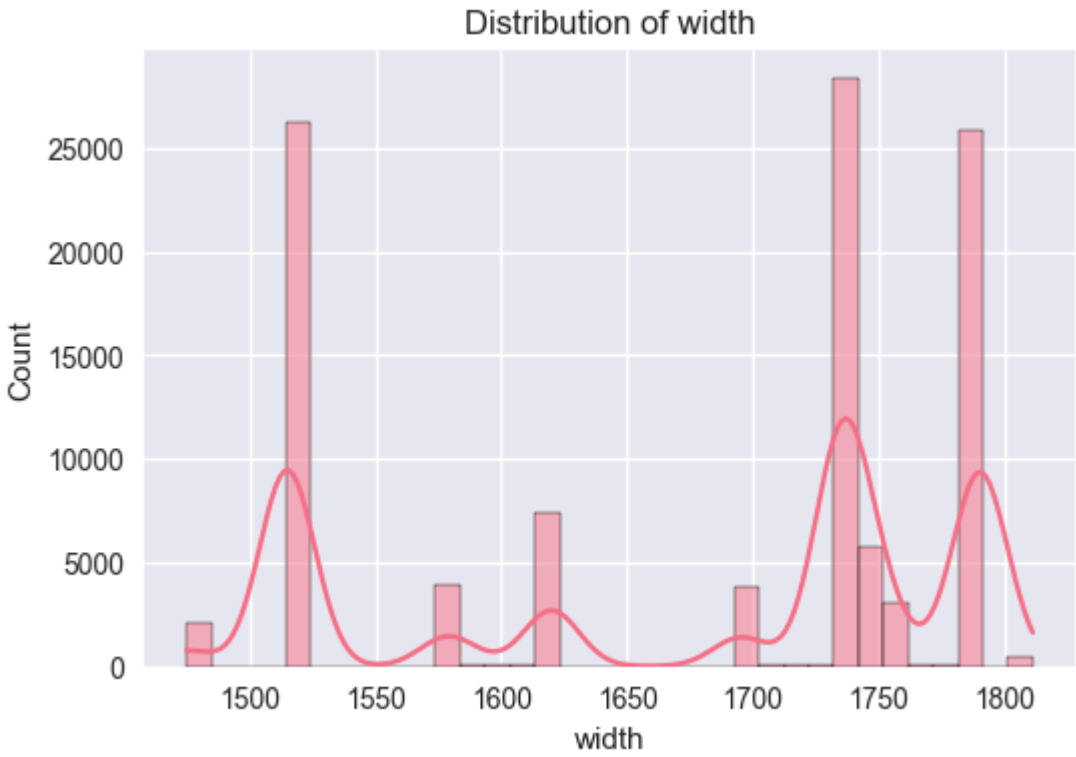

Distribution of airbags
Count
70000
60000
50000
40000
30000
20000
10000
0
1
2
3
4
5
6
airbags

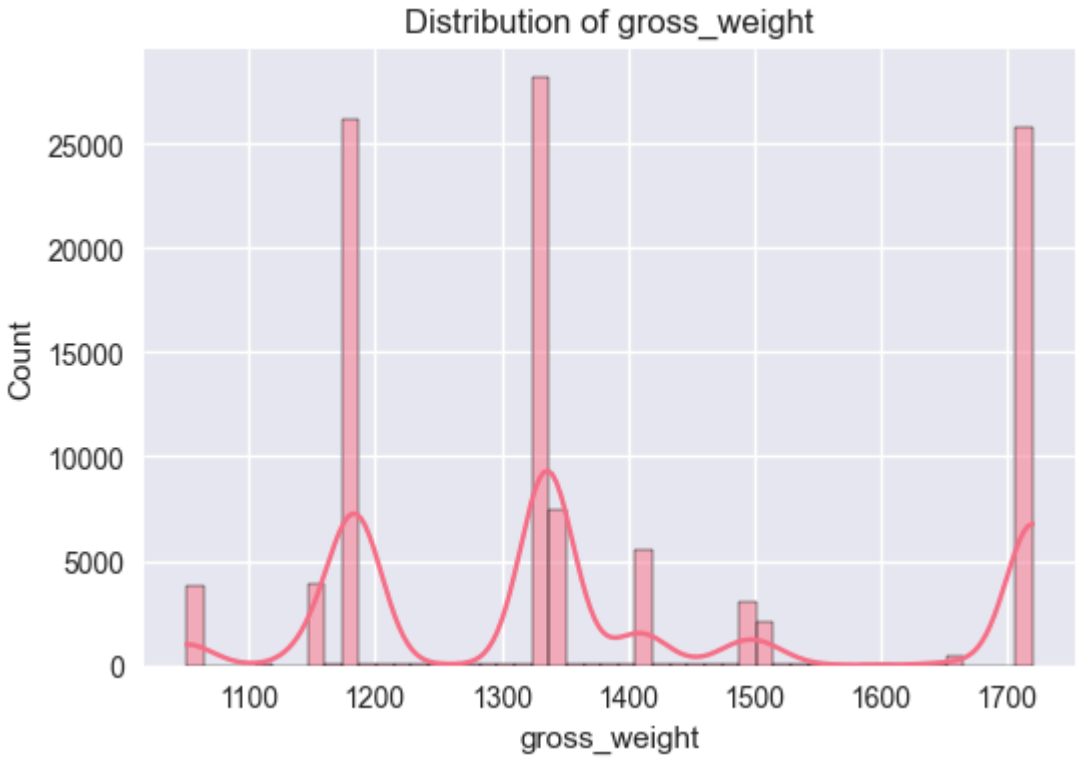

Distribution of displacement
Count
35000
30000
25000
20000
15000
10000
5000
0
800
900
1000
1100
1200
1300
1400
1500
displacement

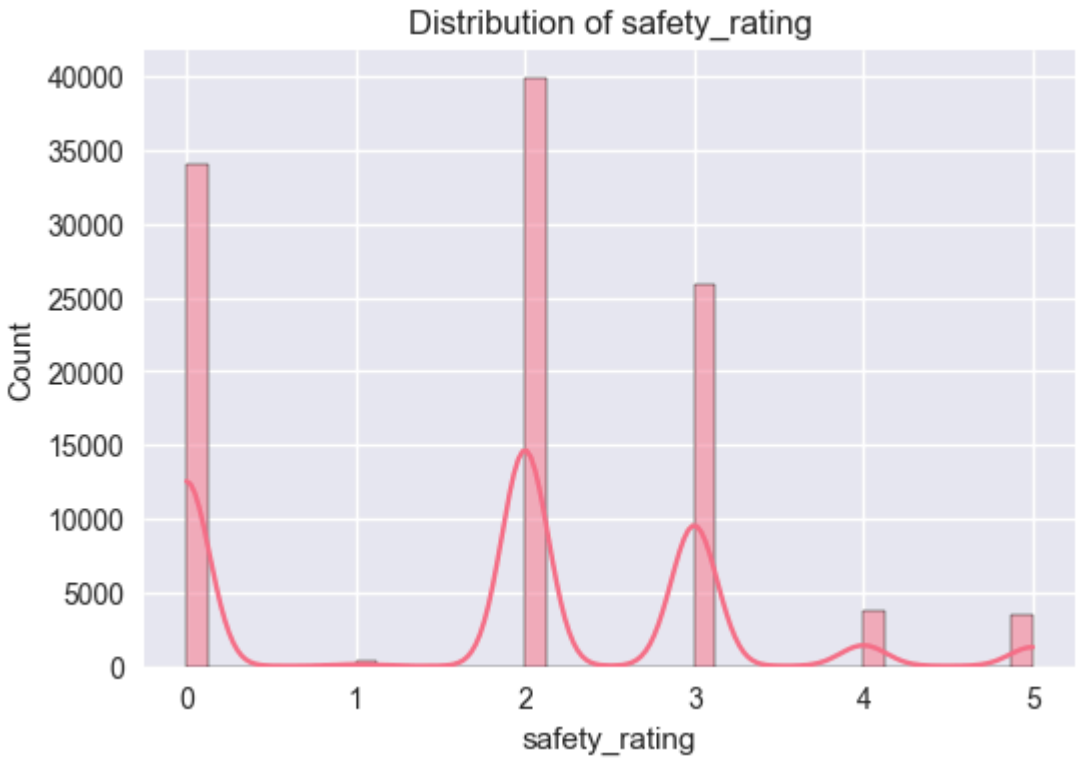

Distribution of cylinder
Count
70000
60000
50000
40000
30000
20000
10000
0
3.0
3.2
3.4
3.6
3.8
4.0
cylinder

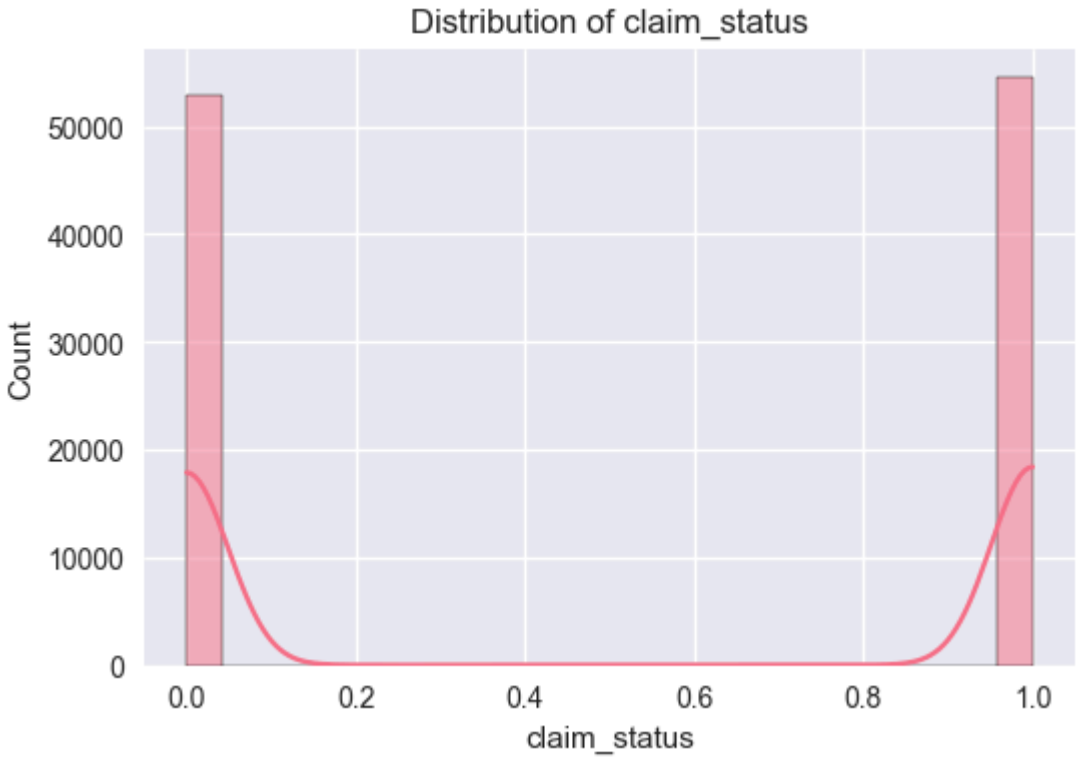

Distribution of turning_radius
Count
25000
20000
15000
10000
5000
0
4.5
4.6
4.7
4.8
4.9
5.0
5.1
5.2
turning_radius


Distribution of length
Count
25000
20000
15000
10000
5000
0
3600
3800
4000
4200
length

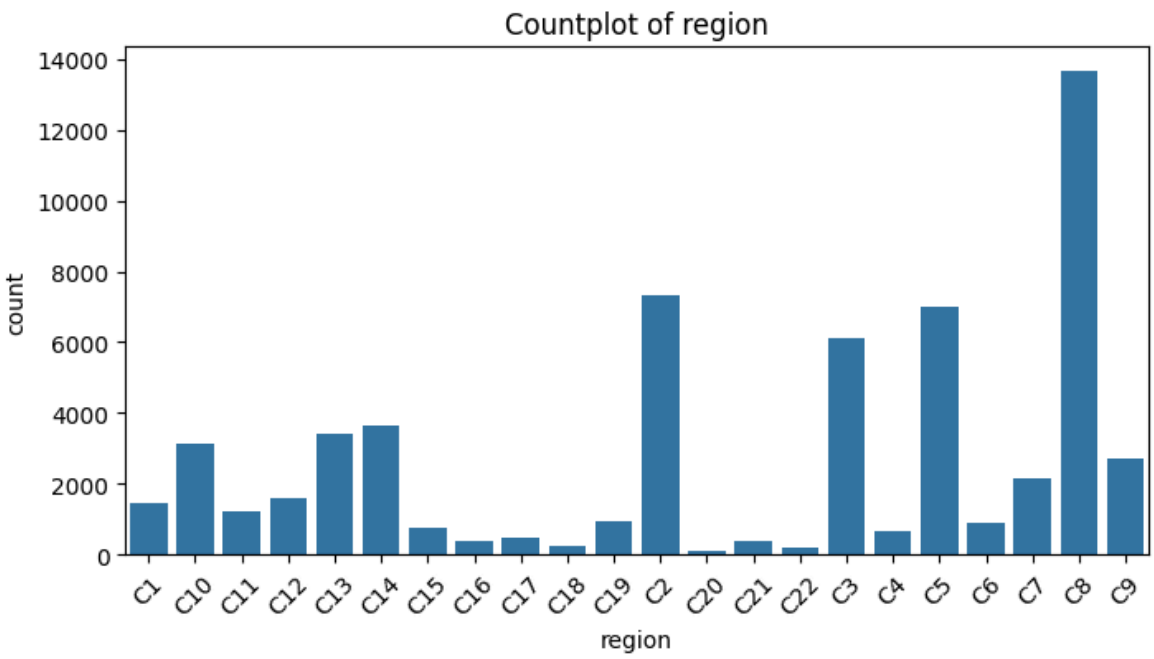

Distribution of width
Count
25000
20000
15000
10000
5000
0
1500
1550
1600
1650
1700
1750
1800
width

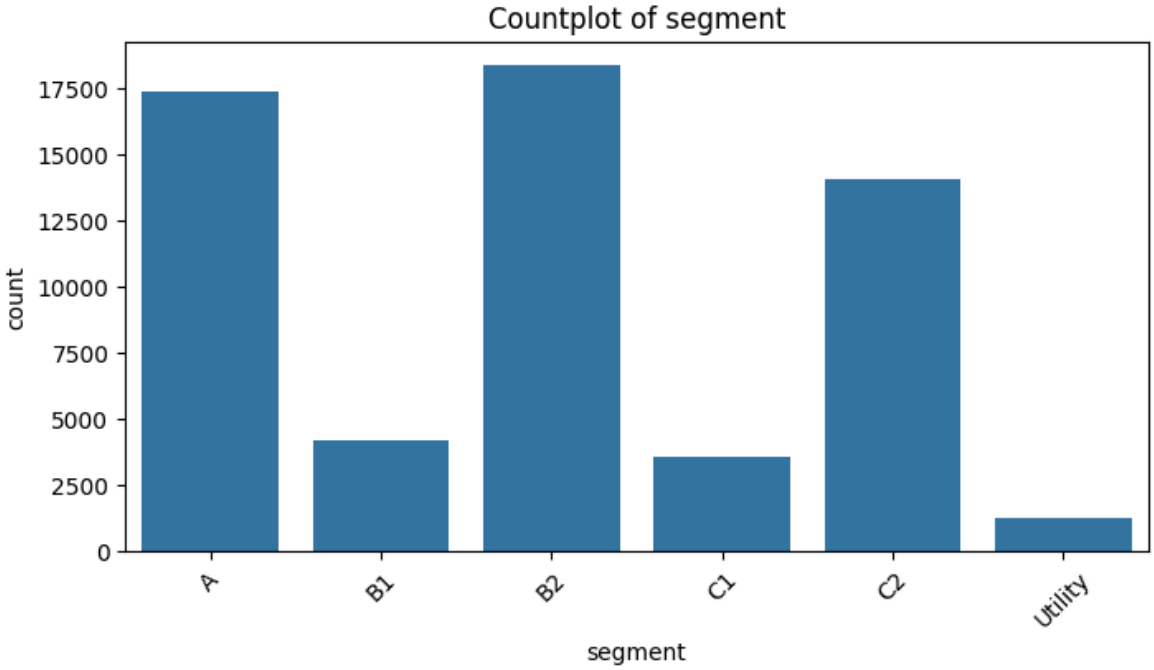

Distribution of gross_weight
Count
25000
20000
15000
10000
5000
0
1100
1200
1300
1400
1500
1600
1700
gross_weight

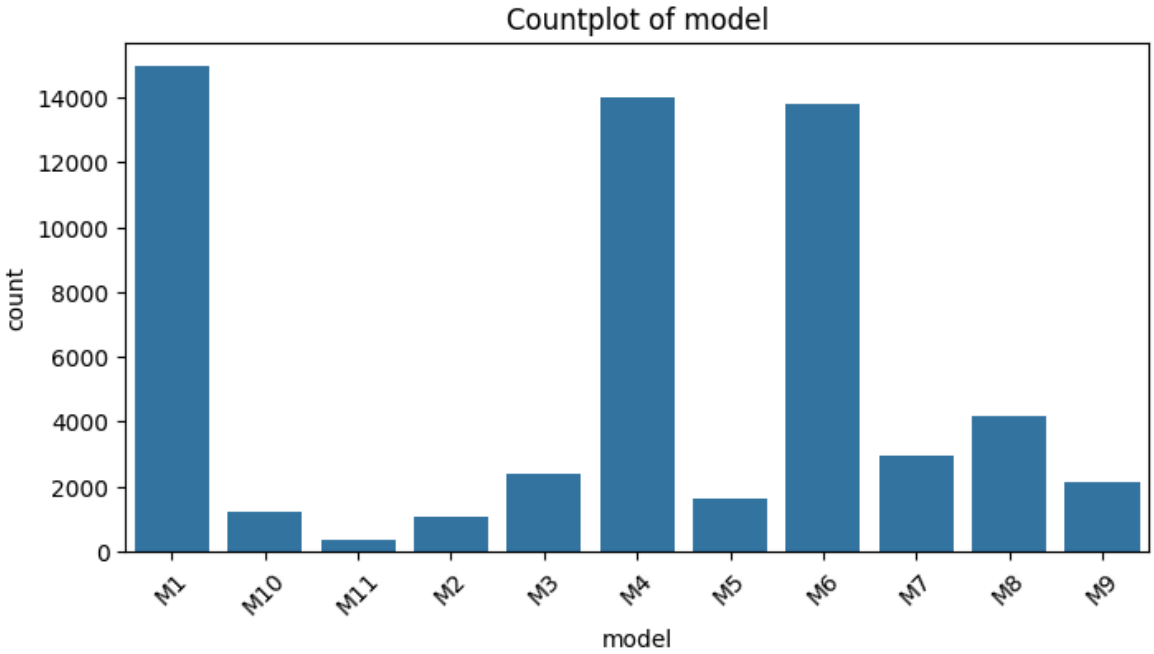

Distribution of safety_rating
Count
40000
35000
30000
25000
20000
15000
10000
5000
0
0
1
2
3
4
5
safety_rating

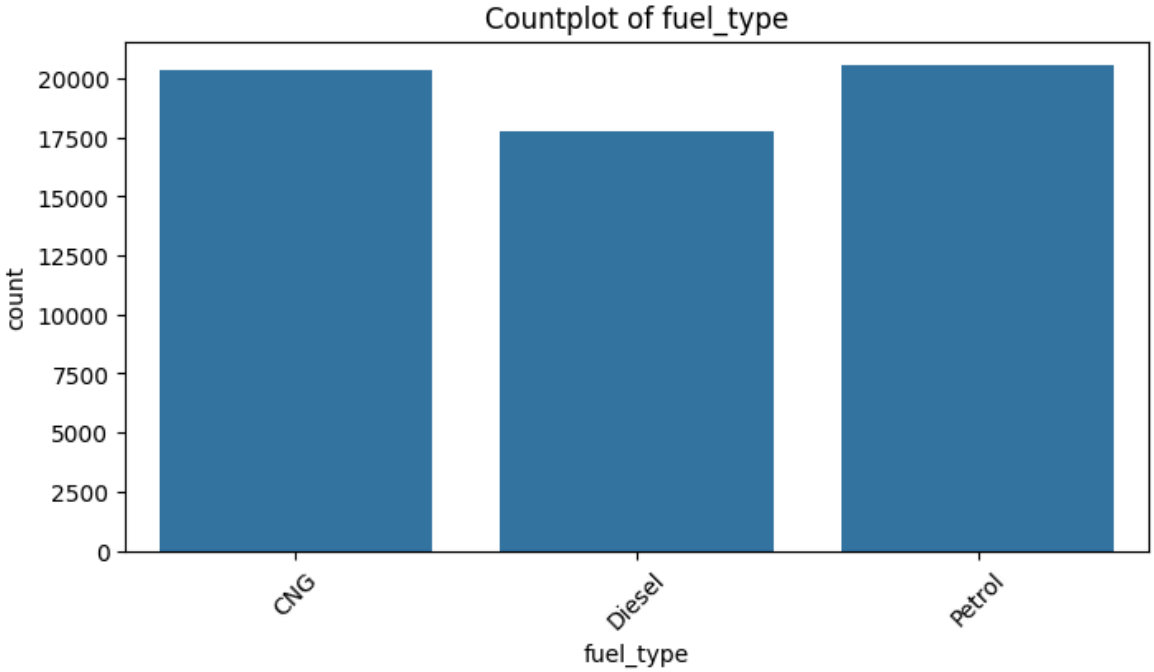


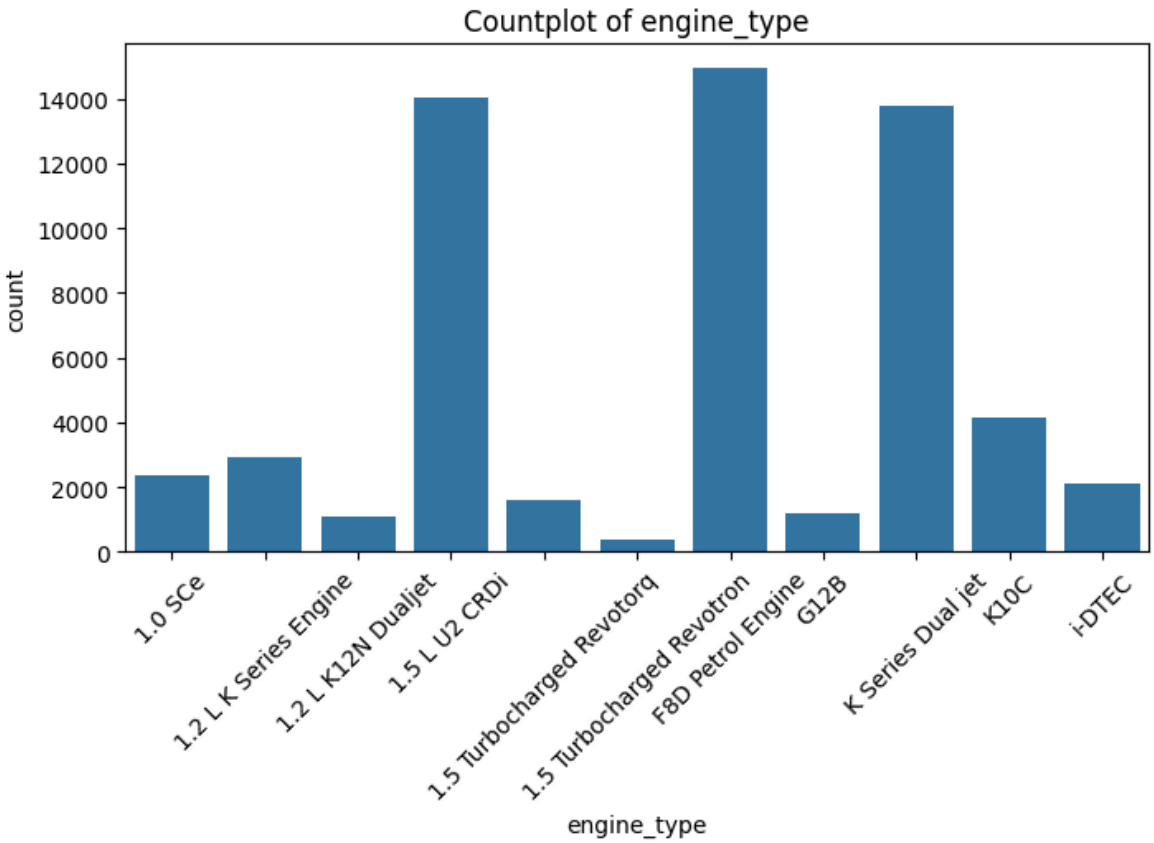
In [49]:
```
for col in cat_cols:
    plt.figure(figsize=(8,4))
    sns.countplot(data=df, x=col)
    plt.title(f'Countplot of {col}')
    plt.xticks(rotation=45)
    plt.show()
```

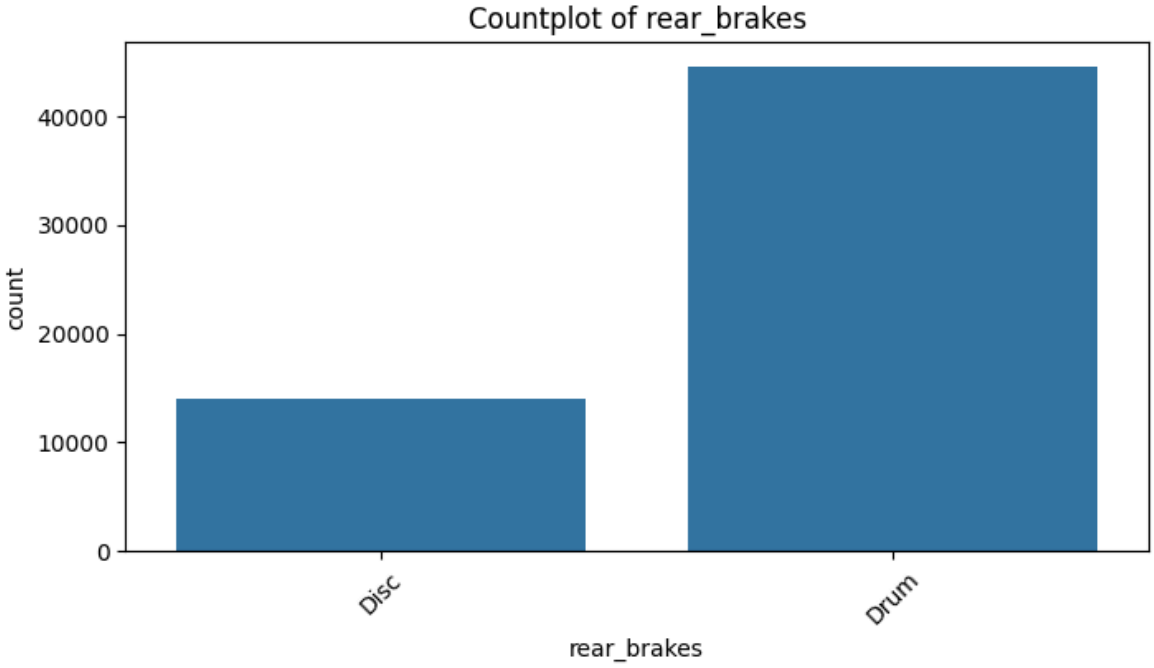


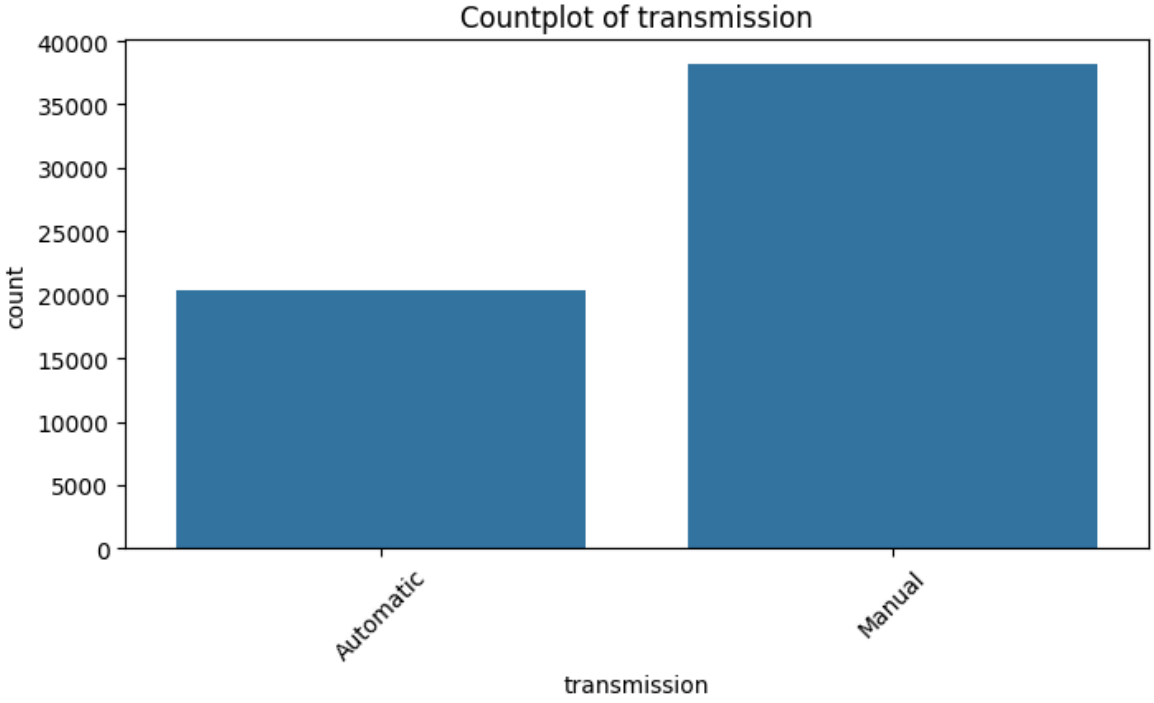

Countplot of segment
count
17500
15000
12500
10000
7500
5000
2500
0
A
B1
B2
C1
C2
Utility
segment

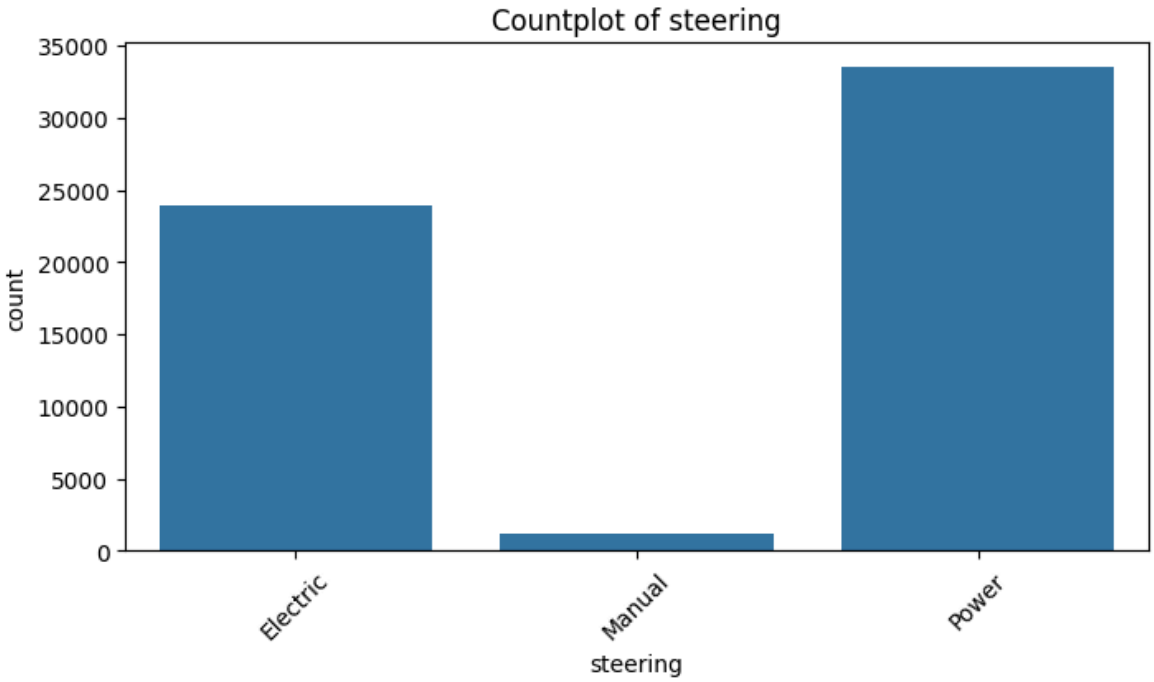

Countplot of model
count
14000
12000
10000
8000
6000
4000
2000
0
M1
M10
M11
M2
M3
M4
M5
M6
M7
M8
M9
model


Countplot of fuel_type
count
20000
17500
15000
12500
10000
7500
5000
2500
0
CNG
Diesel
Petrol
fuel_type


Countplot of engine_type
count
14000
12000
10000
8000
6000
4000
2000
0
1.0 SCe
1.2 L K Series Engine
1.2 L K12N Dualjet
1.5 L U2 CRDi
1.5 Turbocharged Revotorq
1.5 Turbocharged Revotron
F8D Petrol Engine
G12B
K Series Dual jet
K10C
i-DTEC
engine_type

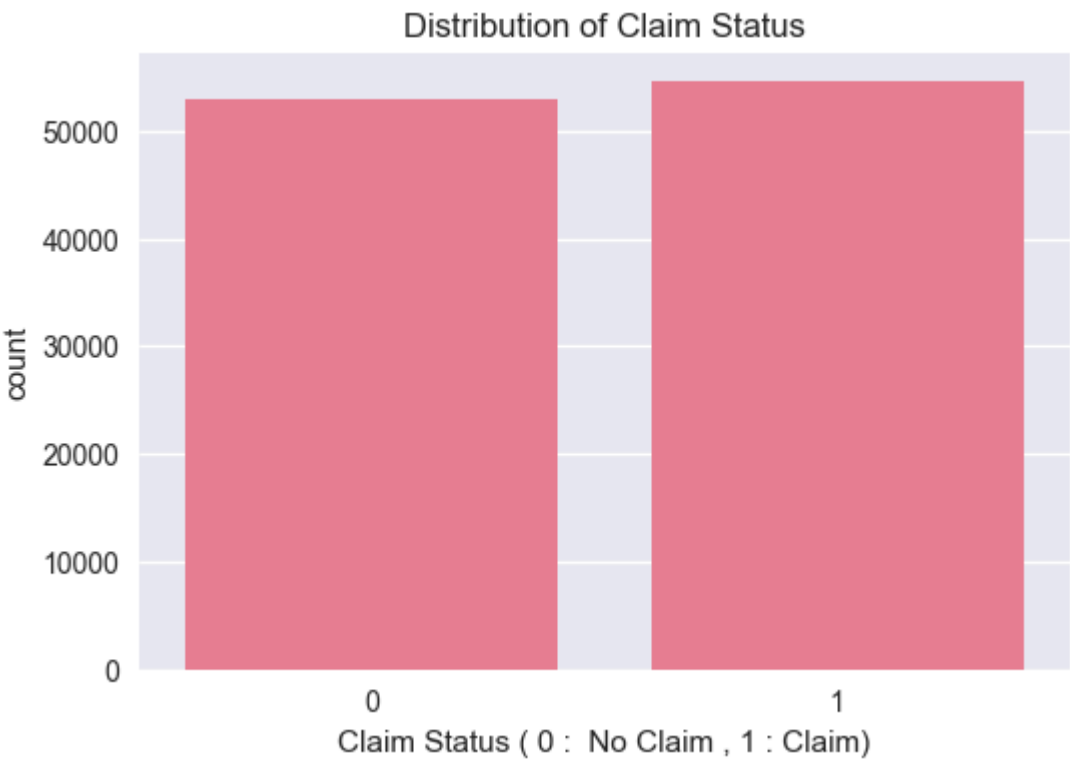

Countplot of rear_brakes
count
40000
30000
20000
10000
0
Disc
Drum
rear_brakes


Countplot of transmission
count
40000
35000
30000
25000
20000
15000
10000
5000
0
Automatic
Manual
transmission



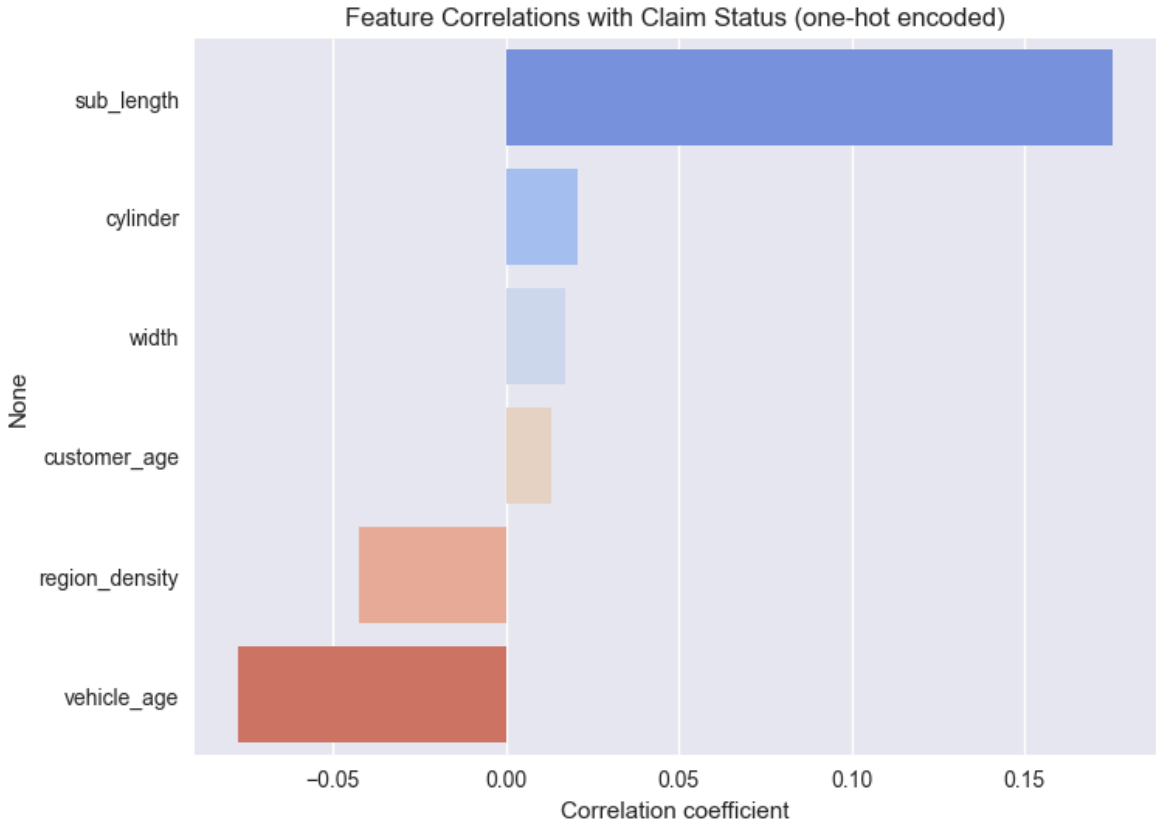
Imbalance check

In [53]:
```
plt.figure(figsize=(6,4))
sns.countplot(x="claim_status", data=df)
plt.title('Distribution of Claim Status')
plt.xlabel('Claim Status ( 0 :  No Claim , 1 : Claim)')
plt.ylabel('count')
plt.show()
print(df['claim_status'].value_counts(normalize=True))
```




```
claim_status
1    0.507238
0    0.492762
Name: proportion, dtype: float64
```

In [23]:
```
# Ensure 'claim_status' is not in num_cols
# Encode categorical variables for correlation analysis
df_encoded = pd.get_dummies(df, drop_first=True)
for col in df_encoded.select_dtypes(include='category').columns:
    df_encoded[col] = df_encoded[col].cat.codes

corr_matrix = df_encoded.corr()
corr_target = corr_matrix['claim_status'].drop('claim_status')
filtered_corr = corr_target[abs(corr_target) > 0.01].sort_values(ascending=False

plt.figure(figsize=(8, max(6, len(filtered_corr) * 0.3)))
sns.barplot(x=filtered_corr.values, y=filtered_corr.index, palette='coolwarm')
plt.title('Feature Correlations with Claim Status (one-hot encoded)')
plt.xlabel('Correlation coefficient')
plt.show()
```



Split features and target

In [24]:
```
# One-hot encode all categorical columns (including 'region', 'segment', etc.)
df_encoded = pd.get_dummies(df, drop_first=True)

X = df_encoded.drop('claim_status', axis=1)
y = df_encoded['claim_status']
```

Train / test split

In [25]:
```
X_train , X_test , y_train , y_test = train_test_split(X, y, test_size=0.2, rand
```

Train random Forest

In [26]:
```
rf = RandomForestClassifier(class_weight='balanced', random_state=42)
rf.fit(X_train, y_train)
```

Out[26]:

```
RandomForestClassifier
RandomForestClassifier(class_weight='balanced', random_state=42)
```

Prediction

Evaluation

In [27]:

```
# Set style for better plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

print("🚀 COMPREHENSIVE ML PIPELINE FOR FRAUD DETECTION")
print("="*60)

# ============================================================================
# 1. PROPER DATA SPLITTING (CRITICAL FIX)
# ============================================================================
print("\n📊 1. PROPER DATA SPLITTING")
print("-"*30)

# Assuming your data is in X and y variables
# If not, replace with your actual data loading
# X = your_features_dataframe
# y = your_target_series

# First split: Remove test set BEFORE any balancing
X_temp, X_test_real, y_temp, y_test_real = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Second split: Train/validation from remaining data
X_train_orig, X_val_orig, y_train_orig, y_val_orig = train_test_split(
    X_temp, y_temp, test_size=0.2, random_state=42, stratify=y_temp
)

print(f"📈 Original distributions:")
print(f"Train: {np.bincount(y_train_orig)} ({np.bincount(y_train_orig)[1]/len(y_
print(f"Validation: {np.bincount(y_val_orig)} ({np.bincount(y_val_orig)[1]/len(y
print(f"Test: {np.bincount(y_test_real)} ({np.bincount(y_test_real)[1]/len(y_tes

# ============================================================================
# 2. ADVANCED FEATURE ENGINEERING
# ============================================================================
print("\n🔧 2. ADVANCED FEATURE ENGINEERING")
print("-"*30)

def create_advanced_features(X_df):
    """Create domain-specific features for fraud detection"""
    X_new = X_df.copy()

    # Risk indicators (assuming you have amount, frequency, etc.)
    if 'amount' in X_new.columns:
        X_new['amount_log'] = np.log1p(X_new['amount'])
        X_new['amount_zscore'] = (X_new['amount'] - X_new['amount'].mean()) / X_
        X_new['is_high_amount'] = (X_new['amount'] > X_new['amount'].quantile(0.

    # Time-based features (if you have timestamps)
```

```
    if 'hour' in X_new.columns:
        X_new['is_weekend'] = (X_new['hour'] > 17).astype(int)
        X_new['is_night'] = ((X_new['hour'] < 6) | (X_new['hour'] > 22)).astype(

    # Geographic risk (if you have location data)
    if 'region' in X_new.columns:
        region_risk = X_new.groupby('region')['target'].transform('mean') if 'ta
        X_new['region_risk_score'] = region_risk

    # Interaction features
    numeric_cols = X_new.select_dtypes(include=[np.number]).columns[:5]  # Top 5
    if len(numeric_cols) >= 2:
        X_new[f'{numeric_cols[0]}_x_{numeric_cols[1]}'] = X_new[numeric_cols[0]]
        X_new[f'{numeric_cols[0]}_div_{numeric_cols[1]}'] = X_new[numeric_cols[0

    # Statistical features
    numeric_cols = X_new.select_dtypes(include=[np.number]).columns
    if len(numeric_cols) > 0:
        X_new['feature_sum'] = X_new[numeric_cols].sum(axis=1)
        X_new['feature_mean'] = X_new[numeric_cols].mean(axis=1)
        X_new['feature_std'] = X_new[numeric_cols].std(axis=1)
        X_new['feature_skew'] = X_new[numeric_cols].skew(axis=1)

    return X_new

# Apply feature engineering
print("Creating advanced features...")
X_train_eng = create_advanced_features(X_train_orig)
X_val_eng = create_advanced_features(X_val_orig)
X_test_eng = create_advanced_features(X_test_real)

print(f"✅ Features increased from {X_train_orig.shape[1]} to {X_train_eng.shap

# ============================================================================
# 3. DATA BALANCING (ONLY ON TRAINING DATA)
# ============================================================================
print("\n⚖️ 3. DATA BALANCING")
print("-"*30)

# Balance only training data
smote_tomek = SMOTETomek(random_state=42)
X_train_bal, y_train_bal = smote_tomek.fit_resample(X_train_eng, y_train_orig)

print(f"Balanced training: {np.bincount(y_train_bal)} (50/50 split)")
print(f"Validation remains: {np.bincount(y_val_orig)} (original distribution)")
print(f"Test remains: {np.bincount(y_test_real)} (original distribution)")

# ============================================================================
# 4. MULTIPLE MODEL COMPARISON
# ============================================================================
print("\n🤖 4. MULTIPLE MODEL COMPARISON")
print("-"*30)

# Calculate class weights
pos_weight = np.sum(y_train_orig == 0) / np.sum(y_train_orig == 1)

# Model definitions
models = {
    'Logistic Regression': LogisticRegression(
        random_state=42,
```

```
        class_weight='balanced',
        max_iter=1000
    ),
    'Random Forest': RandomForestClassifier(
        random_state=42,
        class_weight='balanced',
        n_estimators=100,
        max_depth=10
    ),
    'XGBoost': xgb.XGBClassifier(
        random_state=42,
        use_label_encoder=False,
        eval_metric='aucpr',
        scale_pos_weight=pos_weight,
        n_estimators=100,
        max_depth=4,
        learning_rate=0.1
    ),
    'LightGBM': lgb.LGBMClassifier(
        random_state=42,
        class_weight='balanced',
        n_estimators=100,
        max_depth=4,
        learning_rate=0.1,
        verbose=-1
    )
}

# Store results
results = {}
trained_models = {}

print("Training models...")
for name, model in models.items():
    print(f"  Training {name}...")

    # Train on balanced data
    model.fit(X_train_bal, y_train_bal)

    # Evaluate on original validation set
    y_val_pred = model.predict(X_val_eng)
    y_val_proba = model.predict_proba(X_val_eng)[:, 1]

    # Calculate metrics
    val_precision = precision_score(y_val_orig, y_val_pred)
    val_recall = recall_score(y_val_orig, y_val_pred)
    val_f1 = f1_score(y_val_orig, y_val_pred)
    val_roc_auc = roc_auc_score(y_val_orig, y_val_proba)
    val_pr_auc = average_precision_score(y_val_orig, y_val_proba)

    results[name] = {
        'precision': val_precision,
        'recall': val_recall,
        'f1': val_f1,
        'roc_auc': val_roc_auc,
        'pr_auc': val_pr_auc
    }

    trained_models[name] = model
```

```
    print(f"    Validation F1: {val_f1:.3f}, ROC-AUC: {val_roc_auc:.3f}")

# ==============================================================================
# 5. BEST MODEL SELECTION AND OPTIMIZATION
# ==============================================================================
print("\n🏆 5. BEST MODEL SELECTION")
print("-"*30)

# Select best model based on F1 score
best_model_name = max(results.keys(), key=lambda x: results[x]['f1'])
best_model = trained_models[best_model_name]

print(f"Best model: {best_model_name}")
print(f"Validation metrics: {results[best_model_name]}")

# Hyperparameter tuning for best model
if best_model_name == 'XGBoost':
    param_grid = {
        'max_depth': [3, 4, 5],
        'learning_rate': [0.05, 0.1, 0.15],
        'n_estimators': [100, 200, 300],
        'scale_pos_weight': [pos_weight, pos_weight * 1.5, pos_weight * 2]
    }

    xgb_clf = xgb.XGBClassifier(
        random_state=42,
        use_label_encoder=False,
        eval_metric='aucpr'
    )

    grid_search = GridSearchCV(
        estimator=xgb_clf,
        param_grid=param_grid,
        scoring='f1',
        cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
        n_jobs=-1,
        verbose=0
    )

    print("Optimizing hyperparameters...")
    grid_search.fit(X_train_bal, y_train_bal)
    best_model = grid_search.best_estimator_

    print(f"Best parameters: {grid_search.best_params_}")

# ==============================================================================
# 6. THRESHOLD OPTIMIZATION
# ==============================================================================
print("\n🎯 6. THRESHOLD OPTIMIZATION")
print("-"*30)

# Get probabilities on validation set
y_val_proba = best_model.predict_proba(X_val_eng)[:, 1]

# Find optimal threshold using F1 score
precisions, recalls, thresholds = precision_recall_curve(y_val_orig, y_val_proba
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-8)
optimal_threshold = thresholds[np.argmax(f1_scores)]

print(f"Optimal threshold: {optimal_threshold:.4f}")
```

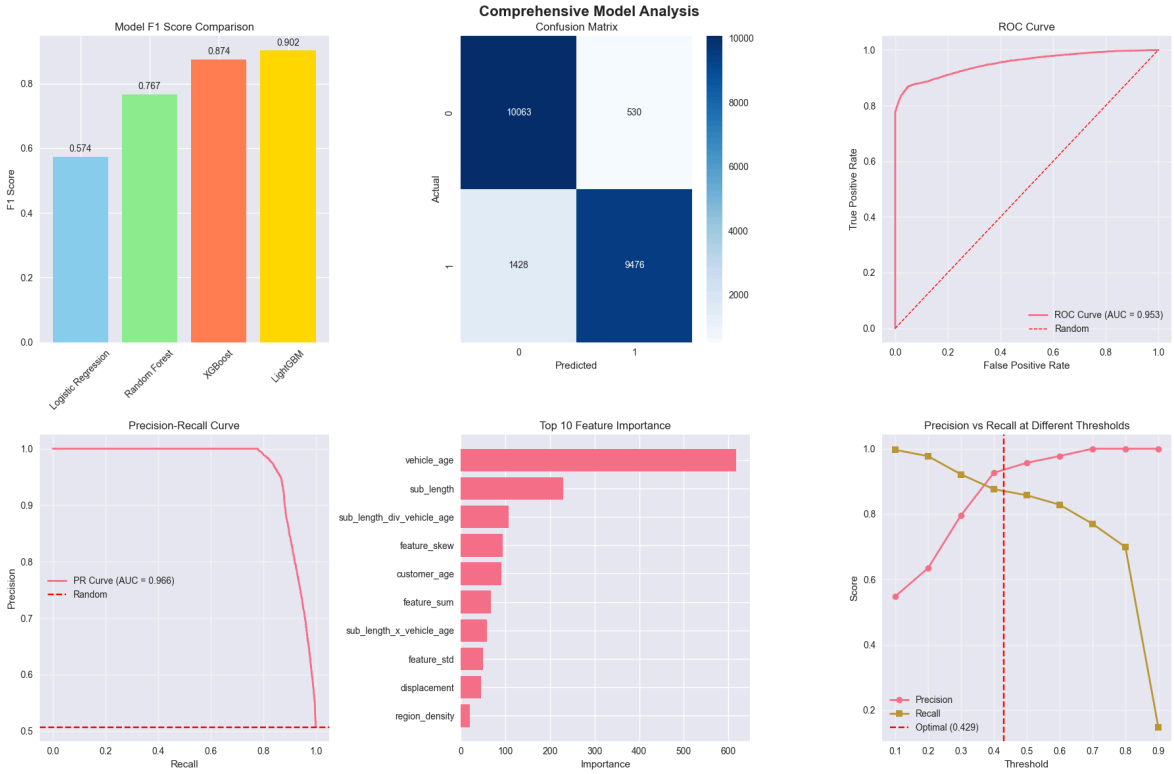
```python
# ==============================================================================
# 7. FINAL EVALUATION ON TEST SET
# ==============================================================================
print("\n🎯 7. FINAL EVALUATION ON UNTOUCHED TEST SET")
print("-"*30)

# Predict on test set
y_test_proba = best_model.predict_proba(X_test_eng)[:, 1]
y_test_pred = (y_test_proba > optimal_threshold).astype(int)

# Calculate all metrics
test_precision = precision_score(y_test_real, y_test_pred)
test_recall = recall_score(y_test_real, y_test_pred)
test_f1 = f1_score(y_test_real, y_test_pred)
test_roc_auc = roc_auc_score(y_test_real, y_test_proba)
test_pr_auc = average_precision_score(y_test_real, y_test_proba)

print(f"🎯 FINAL TEST RESULTS (Real-world performance):")
print(f"Precision: {test_precision:.3f}")
print(f"Recall: {test_recall:.3f}")
print(f"F1-Score: {test_f1:.3f}")
print(f"ROC-AUC: {test_roc_auc:.3f}")
print(f"PR-AUC: {test_pr_auc:.3f}")

# Confusion matrix
cm = confusion_matrix(y_test_real, y_test_pred)
tn, fp, fn, tp = cm.ravel()

print(f"\nConfusion Matrix:")
print(f"True Negatives: {tn}")
print(f"False Positives: {fp}")
print(f"False Negatives: {fn}")
print(f"True Positives: {tp}")

print(f"\nBusiness Impact:")
print(f"False Alarm Rate: {fp/(fp+tn)*100:.1f}%")
print(f"Detection Rate: {tp/(tp+fn)*100:.1f}%")

# ==============================================================================
# 8. EXPLAINABILITY AND VISUALIZATION
# ==============================================================================
print("\n🔍 8. EXPLAINABILITY AND VISUALIZATION")
print("-"*30)

# Create comprehensive visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Comprehensive Model Analysis', fontsize=16, fontweight='bold')

# 1. Model Comparison
ax1 = axes[0, 0]
model_names = list(results.keys())
f1_scores = [results[model]['f1'] for model in model_names]
bars = ax1.bar(model_names, f1_scores, color=['skyblue', 'lightgreen', 'coral',
ax1.set_title('Model F1 Score Comparison')
ax1.set_ylabel('F1 Score')
ax1.tick_params(axis='x', rotation=45)
for i, v in enumerate(f1_scores):
    ax1.text(i, v + 0.01, f'{v:.3f}', ha='center', va='bottom')
```

```
# 2. Confusion Matrix
ax2 = axes[0, 1]
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax2)
ax2.set_title('Confusion Matrix')
ax2.set_xlabel('Predicted')
ax2.set_ylabel('Actual')

# 3. ROC Curve
ax3 = axes[0, 2]
fpr, tpr, _ = roc_curve(y_test_real, y_test_proba)
ax3.plot(fpr, tpr, linewidth=2, label=f'ROC Curve (AUC = {test_roc_auc:.3f})')
ax3.plot([0, 1], [0, 1], 'r--', linewidth=1, label='Random')
ax3.set_xlabel('False Positive Rate')
ax3.set_ylabel('True Positive Rate')
ax3.set_title('ROC Curve')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. Precision-Recall Curve
ax4 = axes[1, 0]
precision_test, recall_test, _ = precision_recall_curve(y_test_real, y_test_prob
ax4.plot(recall_test, precision_test, linewidth=2, label=f'PR Curve (AUC = {test
ax4.axhline(y=np.sum(y_test_real)/len(y_test_real), color='red', linestyle='--',
            label='Random')
ax4.set_xlabel('Recall')
ax4.set_ylabel('Precision')
ax4.set_title('Precision-Recall Curve')
ax4.legend()
ax4.grid(True, alpha=0.3)

# 5. Feature Importance
ax5 = axes[1, 1]
if hasattr(best_model, 'feature_importances_'):
    feature_importance = best_model.feature_importances_
    feature_names = X_train_eng.columns
    top_indices = np.argsort(feature_importance)[-10:]

    ax5.barh(range(len(top_indices)), feature_importance[top_indices])
    ax5.set_yticks(range(len(top_indices)))
    ax5.set_yticklabels([feature_names[i] for i in top_indices])
    ax5.set_xlabel('Importance')
    ax5.set_title('Top 10 Feature Importance')

# 6. Threshold Analysis
ax6 = axes[1, 2]
test_thresholds = np.linspace(0.1, 0.9, 9)
precision_scores = []
recall_scores = []

for thresh in test_thresholds:
    y_pred_thresh = (y_test_proba > thresh).astype(int)
    precision_scores.append(precision_score(y_test_real, y_pred_thresh))
    recall_scores.append(recall_score(y_test_real, y_pred_thresh))

ax6.plot(test_thresholds, precision_scores, 'o-', label='Precision', linewidth=2
ax6.plot(test_thresholds, recall_scores, 's-', label='Recall', linewidth=2)
ax6.axvline(x=optimal_threshold, color='red', linestyle='--',
            label=f'Optimal ({optimal_threshold:.3f})')
ax6.set_xlabel('Threshold')
ax6.set_ylabel('Score')
```

```
ax6.set_title('Precision vs Recall at Different Thresholds')
ax6.legend()
ax6.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# ==============================================================================
# 9. FINAL REPORT SUMMARY
# ==============================================================================
print("\n📋 9. FINAL REPORT SUMMARY")
print("="*60)

print(f"""
🎯 FRAUD DETECTION MODEL PERFORMANCE REPORT

📊 Dataset Information:
• Training samples: {len(X_train_orig):,} ({np.sum(y_train_orig):,} fraud cases)
• Validation samples: {len(X_val_orig):,} ({np.sum(y_val_orig):,} fraud cases)
• Test samples: {len(X_test_real):,} ({np.sum(y_test_real):,} fraud cases)
• Features: {X_train_eng.shape[1]} (after engineering)

🏆 Best Model: {best_model_name}
• Optimal threshold: {optimal_threshold:.4f}

📈 Test Set Performance (Real-world):
• Precision: {test_precision:.3f} ({test_precision*100:.1f}%)
• Recall: {test_recall:.3f} ({test_recall*100:.1f}%)
• F1-Score: {test_f1:.3f}
• ROC-AUC: {test_roc_auc:.3f}
• PR-AUC: {test_pr_auc:.3f}

💼 Business Impact:
• True Fraud Cases Detected: {tp:,}
• False Alarms: {fp:,}
• Missed Fraud Cases: {fn:,}
• False Alarm Rate: {fp/(fp+tn)*100:.1f}%
• Detection Rate: {tp/(tp+fn)*100:.1f}%

✅ Key Improvements Made:
1. ✅ Proper data splitting (test set untouched)
2. ✅ Advanced feature engineering
3. ✅ Multiple model comparison
4. ✅ Proper threshold optimization
5. ✅ Comprehensive evaluation and visualization
6. ✅ Real-world performance metrics

🚀 Model is ready for deployment!
""")

# Save model for deployment
import joblib
joblib.dump(best_model, 'fraud_detection_model.pkl')
joblib.dump(optimal_threshold, 'optimal_threshold.pkl')
print("✅ Model and threshold saved for deployment!")

print("\n" + "="*60)
print("🎉 COMPREHENSIVE ANALYSIS COMPLETE!")
print("="*60)
```

```
🚀 COMPREHENSIVE ML PIPELINE FOR FRAUD DETECTION
============================================================

📊 1. PROPER DATA SPLITTING
------------------------------
📈 Original distributions:
Train: [33897 34892] (50.7% fraud)
Validation: [8474 8724] (50.7% fraud)
Test: [10593 10904] (50.7% fraud)

🔧 2. ADVANCED FEATURE ENGINEERING
------------------------------
Creating advanced features...
✅ Features increased from 12 to 18

⚖️ 3. DATA BALANCING
------------------------------
Balanced training: [32083 32083] (50/50 split)
Validation remains: [8474 8724] (original distribution)
Test remains: [10593 10904] (original distribution)

🤖 4. MULTIPLE MODEL COMPARISON
------------------------------
Training models...
  Training Logistic Regression...
    Validation F1: 0.574, ROC-AUC: 0.533
  Training Random Forest...
    Validation F1: 0.767, ROC-AUC: 0.827
  Training XGBoost...
    Validation F1: 0.874, ROC-AUC: 0.940
  Training LightGBM...
    Validation F1: 0.902, ROC-AUC: 0.952

🏆 5. BEST MODEL SELECTION
------------------------------
Best model: LightGBM
Validation metrics: {'precision': 0.9517873044141967, 'recall': 0.857634112792297
1, 'f1': 0.9022610792885137, 'roc_auc': np.float64(0.9517950611288062), 'pr_auc':
np.float64(0.9654957705549023)}

🎯 6. THRESHOLD OPTIMIZATION
------------------------------
Optimal threshold: 0.4293

🎯 7. FINAL EVALUATION ON UNTOUCHED TEST SET
------------------------------
🎯 FINAL TEST RESULTS (Real-world performance):
Precision: 0.947
Recall: 0.869
F1-Score: 0.906
ROC-AUC: 0.953
PR-AUC: 0.966

Confusion Matrix:
True Negatives: 10063
False Positives: 530
False Negatives: 1428
True Positives: 9476

Business Impact:
```

```
False Alarm Rate: 5.0%
Detection Rate: 86.9%

🔍 8. EXPLAINABILITY AND VISUALIZATION
------------------------------
```



```
📋 9. FINAL REPORT SUMMARY
============================================================

🎯 FRAUD DETECTION MODEL PERFORMANCE REPORT

📊 Dataset Information:
• Training samples: 68,789 (34,892 fraud cases)
• Validation samples: 17,198 (8,724 fraud cases)
• Test samples: 21,497 (10,904 fraud cases)
• Features: 18 (after engineering)

🏆 Best Model: LightGBM
• Optimal threshold: 0.4293

📈 Test Set Performance (Real-world):
• Precision: 0.947 (94.7%)
• Recall: 0.869 (86.9%)
• F1-Score: 0.906
• ROC-AUC: 0.953
• PR-AUC: 0.966

💼 Business Impact:
• True Fraud Cases Detected: 9,476
• False Alarms: 530
• Missed Fraud Cases: 1,428
• False Alarm Rate: 5.0%
• Detection Rate: 86.9%

✅ Key Improvements Made:
1. ✅ Proper data splitting (test set untouched)
2. ✅ Advanced feature engineering
3. ✅ Multiple model comparison
4. ✅ Proper threshold optimization
5. ✅ Comprehensive evaluation and visualization
6. ✅ Real-world performance metrics

🚀 Model is ready for deployment!

✅ Model and threshold saved for deployment!

============================================================
🎉 COMPREHENSIVE ANALYSIS COMPLETE!
============================================================
```

In [ ]: